13章

現実世界のドメイン駆動設計

　ドメイン駆動設計の技法を使って、事業活動を深く理解し、業務知識を共有し、設計の意思決定を行うための考え方とやり方を説明してきました。この知識を実際のソフトウェア開発に活かせたら、どんなに楽しいか想像してみてください。これから始まる新規プロジェクトの様子を想像してみましょう。

チーム全員がドメイン駆動設計を熟知し、最初から全員が役に立つモデルの探求に全力をつくし、当然のことながら、全員が同じ言葉を忠実に使っています。プロジェクトが進むにつれ、区切られた文脈の境界が明確になり、その境界は事業活動のさまざまなモデルをしっかりと保護しています。事業戦略と整合した実装方法が選択され、コードベースは体系的に整理されています。つまり、ソースコードが同じ言葉を語り、モデルの複雑さをうまく扱うための技法があちこちにちりばめられています。

　残念ながら、これは夢物語です。目を覚ましましょう。

　こういう理想的な状況でドメイン駆動設計に取り組めるとしたら、それは宝くじに当たったようなものです。もちろん、そういう可能性はゼロではありません。しかし現実には難しいでしょう。残念なことですが、多くの人は、ドメイン駆動設計を適用できるのは、新規の開発案件でドメイン駆動設計に熟達したメンバーがそろっている場合、という理想的な条件を満たしていることが必要だと誤解しています。皮肉なことに、ドメイン駆動設計が大きな効果をもたらすのは、すでに稼働しているソフトウェアに取り組んだ場合です。つまり、事業への貢献が実証されていて、積み重なった技術的負債と膨れ上がった設計の乱雑さを刷新する必要があるシステムこそ、ドメイン駆動設計で取り組む価

値がもっとも高いのです。偶然にも、多くの技術者が取り組んでいるのは、このような、変更がやっかいで危険な大きな泥団子のコードベースです。

ドメイン駆動設計についてありがちな誤解に「すべてかゼロか」の二極化思考があります。ドメイン駆動設計のすべての技法を使うことがドメイン駆動設計であり、一部の技法だけを使うのはドメイン駆動設計ではない、というとらえ方です。そんなことはありません。ドメイン駆動設計のあらゆる側面を理解することは簡単なことではありません。ましてや、すべての技法を習得して実践することは、おそらく不可能でしょう。幸運なことに、すべての技法を取り入れなくても、ドメイン駆動設計は価値を生み出します。このことは、ドメイン駆動設計を既存システムに適用する場合は特に重要です。既存システムの修正や拡張に取り組んでいる場合、限られた時間の中であらゆる技法を取り入れることは不可能です。

この章では、変更がやっかいで危険になった既存システムの修正や拡張という、理想とはほど遠い状況で、ドメイン駆動設計の考え方とやり方を取り入れていくための、戦略的なやり方を学びます。

13.1　戦略的な分析

ドメイン駆動設計の技法について、この本で説明してきた順番を思い出してください。ドメイン駆動設計を導入する時に最初にやるべきことは次の二つです。

- 事業活動を理解する
- システムの構造が現在どうなっているかを理解する

13.1.1　事業活動を理解する

まず自社の事業とは何かを特定します。

- 事業活動を展開している領域はどこですか？
- 顧客は誰ですか？
- 顧客に提供しているサービス、あるいは価値は何ですか？
- 競合している企業や製品は何ですか？

この四つの質問に答えることで、自社の事業活動の基本目標を理解します。次に事業活動の詳細にズームインします。事業目標を達成するための事業活動

の構成要素、つまり業務領域を探します。

よい手がかりは自社の組織図です。つまり自社の部門構成を把握することがよい出発点になります。自社が存続し発展していくために、それぞれの部門がどのように協力しているかを分析します。

さらに、それぞれの業務領域のカテゴリーを特定するための手がかりを探します。

13.1.1.1　中核の業務領域

中核の業務領域は、競合する他社とどうやって差別化しているかを探すことで明らかになります。

- 競合相手が持っていない独自のノウハウを持っていますか？たとえば自社開発したアルゴリズムや特許などの知的財産です。
- 競争優位、つまり中核の業務領域は技術的なものとは限りません。技術とは異なる領域で何らかの競争優位性を持っていますか？たとえば、優秀な人材を採用する能力や、独創的なデザインを生み出す能力です。

中核の業務領域を特定する方法として、強力だが、ありがたくない経験則があります。それは大きな泥団子です。ソフトウェア開発者であれば見るのも触るのもいやでしょうが、システムでもっとも入り組んだ設計になっている場所は、中核の業務領域である可能性が高いでしょう。こういう部分の全面的な書き換えは、大きな事業リスクです。経営的に、そういう選択はできません。重要なことは、この部分が一般的な業務領域ではないことです。つまり、パッケージソフトや外部サービスに置き換えることは不可能です。そして、こういう部分の修正や拡張は、必ず事業リスクが伴います。

13.1.1.2　一般的な業務領域

一般的な業務領域を特定するには、パッケージ製品、外部サービス、あるいはオープンソースソフトウェアを探してみましょう。**第1章**で学んだように、一般的な業務領域は、競合他社も同じ製品やサービスを利用できる業務領域です。そして、他社が同じ製品やサービスを導入しても、自社の不利益にはなりません。

13.1.1.3　補完的な業務領域

補完的な業務領域を特定するには、パッケージ製品や外部サービスに置き換えることはできないが、かといって競争優位を直接生み出すことはないソフトウェアコンポーネントを探します。もしその部分のソースコードが粗削りのままだとしたら、変更の機会が少ないため、技術者が関心を持つことがほとんどなかった部分です。中核の業務領域とは違い、まずい設計のままでも大きな問題にはならない部分です。

あらゆる補完的な業務領域を洗い出す必要はありません。そのやり方は実践的ではないし、中規模の企業だとしても、おそらく不可能でしょう。そのかわりに、事業活動全体を俯瞰してください。そして現在取り組んでいるソフトウェアシステムにもっとも関係する業務領域に注目してください。

13.1.2　既存システムの構造を調べる

業務領域がわかってきたら、次のステップとして、現在のシステムはどのような解決方法を選択し、どのような設計判断をしてきたかを調査します。まずシステム全体の構造を調べます。既存の構造は必ずしもドメイン駆動設計の区切られた文脈ではありません。むしろ、開発の都合でサブシステムに分割した時の境界線でしょう。

注目すべきは、それぞれのサブシステムのライフサイクルの違いです。すべてのサブシステムを一つのソースコードリポジトリ（モノレポ）で管理している場合、あるいは、巨大な一枚岩のコードベースとなっている場合でも、他のサブシステムと切り離して、修正、拡張、テスト、そして配置ができそうなサブシステムがないか確認してみましょう。

13.1.2.1　業務ロジックの実装方法を確認する

システムのコンポーネントごとに、どんな業務領域が含まれているかを調べます。そして、コンポーネントごとの設計方針を確認します。つまり、業務ロジックの実装方法に何を選択し、システム構築の技術方式に何を選択しているかです。

現在の実装方法は、解決すべき課題の複雑さと合致しているでしょうか。設計を洗練させるべき場所がないでしょうか。あるいは、手を抜いたり、既成の製品やサービスを利用したりできそうな業務領域があるでしょうか。こういう問いかけによって集めた情報を、設計方針と実装方法の改善に活用してください。

13.1.2.2　設計の基本方針を評価する

システム全体の構造を俯瞰して得た知識を使って、既存システムのコンポーネントを区切られた文脈のように扱って、現在の構造を文脈の地図として可視化してみましょう。システムのコンポーネントどうしのつながり方を、**第4章**で説明した区切られた文脈の連係方法の分類を使って、特定し追跡してください。

最後に、ドメイン駆動設計の観点から、この文脈の地図を分析し、全体の構造を評価してみましょう。次のような不適切な関係になっていないかを確認します。

- 同じコンポーネントに複数のチームが関わっている
- 中核の業務領域の実装が重複している
- 中核の業務領域の開発を外部に依頼している
- 統合が失敗しがちで摩擦が発生している
- 外部サービスや既存システムに影響された、ぎこちないモデルになっている

こういう洞察が、設計の見直しを検討するよい出発点になります。しかし、その見直しの前に、課題（事業活動）とその解決策（現在の設計）についての知識を得た上で、失われた業務知識を探してみましょう。**第11章**で検討したように、さまざまな理由で業務知識は失われます。業務ロジックが複雑で事業への影響も大きい中核の業務領域では、業務知識の消失は深刻な問題です。もしそういう状況であれば、関係者で集まってイベントストーミングを実施して、失われた業務知識を修復してください。また、イベントストーミングをやりながら、開発に関わる全員が使う同じ言葉を育ててください。

13.2　設計改善の基本方針

この機会に「システム全体を一から書き直し、すべての設計とコードをきれいにしよう」という挑戦が成功することは、ほとんどないでしょう。そして、そういう技術視点のシステム再構築を経営陣が支持することは、まずありえません。

既存システムの設計を改善する安全なやり方は、「大きな絵」を描き、「小さく始める」ことです。エリック・エヴァンスが書いているように、大規模なシステムのすべてをうまく設計することはできません。それが現実です。ですか

ら、どこを重点的に改善するかを戦略的に判断しなければなりません。この戦略的判断の準備として、システムを複数の業務領域に切り分けます。その境界は、コードベースを物理的に分割するような、本格的な区切られた文脈である必要はありません。そうではなく、論理的な境界（名前空間、モジュール、パッケージなど）と業務領域の境界を対応させるところから始めます（図13‐1）。

図13-1　技術視点の区切り方を業務視点の区切り方に再構成する

システムのモジュール構成の変更は、比較的安全なリファクタリングです。業務ロジックの変更ではありません。型（クラス宣言やインターフェース宣言）を配置する場所を移動して、わかりやすい構造に改善します。とはいえ、DLL（ダイナミックロードライブラリ）やリフレクションなど、完全修飾された型名への参照を壊さないように注意してください。

それに加えて、異なるコードベースに記述された業務ロジックを追跡します。データベースのストアードプロシージャやサーバーレスファンクションとして記述されている業務ロジックを特定します。そういう異なる実行環境用に記述された業務ロジックに対しても、新たに定義した業務領域の境界（名前空間）を適用します。たとえば、業務ロジックをストアードプロシージャで実装している場合、どのモジュールの配下にあるプロシージャであるかを明確にするために、プロシージャの名前を変更します。あるいは、モジュールに対応するスキーマをデータベースに追加して、ストアードプロシージャをそちらに移動します。

13.2.1　基本方針の改善

第10章で検討したように、早い段階からシステムをできるだけ小さな文脈に区切るのは危険です。区切られた文脈とマイクロサービスの詳細は、次の章で検討します。ここでは、論理的な境界を物理的な境界に変換することで、大き

な価値を生み出す場所を探してみましょう。図13‐2は、論理的な境界を物理的な境界に変えることで、区切られた文脈を抽出する手順です。

次の点を検討してみましょう。

- 一つのコードベースを複数のチームが変更していますか？もしそうであれば、チームごとに区切られた文脈を定義して、それぞれのチームの開発ライフサイクルを切り離してください。
- 競合するモデルが、異なるコンポーネントによって使われていませんか？もしそうであれば、競合しているモデルをそれぞれ別の区切られた文脈に移動してください。

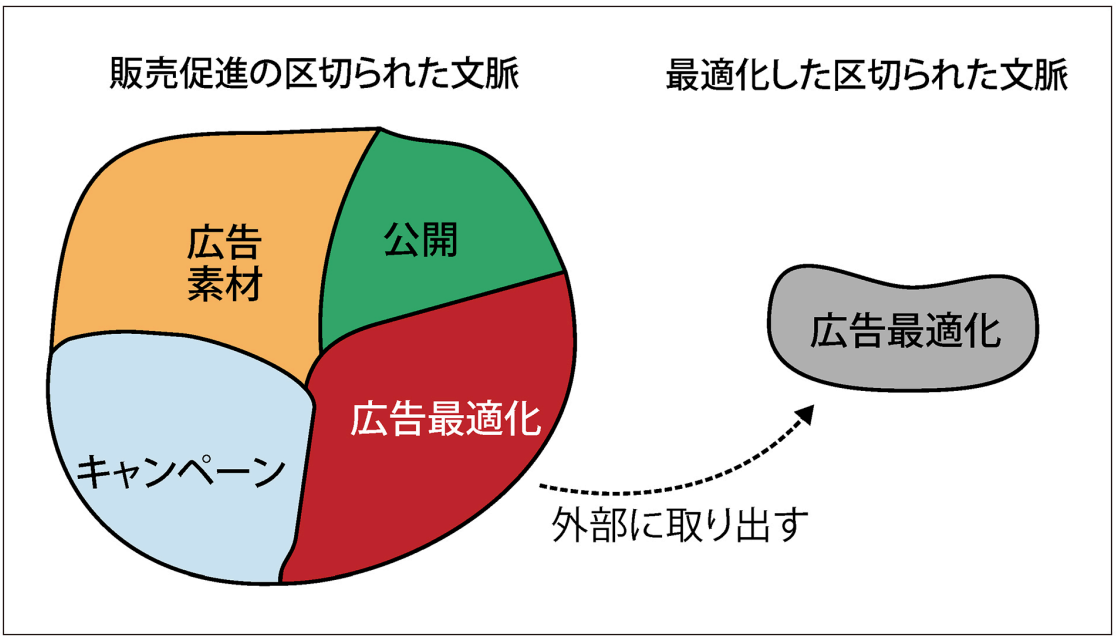

図13-2　論理的な境界を物理的な境界に変更して新たな区切られた文脈を抽出する

最低限必要な区切られた文脈の境界が定まったら、文脈どうしの関係と連係方法を調べてください。異なる区切られた文脈を担当するチームが、どのように意図を伝え協力しているかを確かめます。特に、チーム間のやりとりが臨機応変なやり方、あるいはモデルの共有に近いやり方をしている場合は、チーム間で目標を共有できているか、そして、適切に協力できているかを確認してください。

文脈どうしを連係するさまざまな方法の目的、つまりどんな課題を解決しようとしているかに注意してください。

利用者と供給者の関係

第11章で検討したように、組織が小さい時にはうまくいっていた意図の伝達や協力が、組織が大きくなるとうまくいかなくなることがあります。チーム間が良きパートナーの関係で進めることを想定して設計されたコンポーネントを調べ、そういう協力関係が維持できていない場合、利用者と供給者の関係（従属、モデル変換装置、共用サービス）に変更してください。

モデル変換装置

区切られた文脈を既存システムから保護するには、モデル変換装置が役に立つでしょう。特に既存システムのモデルが下流のコンポーネントに悪い影響を及ぼしている場合です。

あるいは、上流コンポーネントの公開インターフェースが頻繁に変更される場合、下流の区切られた文脈を保護するために、モデル変換装置が役に立ちます。

共用サービス

コンポーネントの実装を変更すると、その変更の影響がシステム全体にさざ波のように広がり、コンポーネントの多くの利用者に影響が及ぶ場合、共用サービスを作ることを検討してください。つまり、外部に公開するAPIと内部のモデルを分離します。

互いに独立

大きな組織では特にそうですが、複数のチームで共通機能の開発や修正に取り組む場合、摩擦や衝突が起きることがあります。そういう「争いの種」が事業活動にとって重要でない場合、つまり中核の業務領域でない場合、それぞれの開発チームが互いに独立して開発を進め衝突を避けるのも選択肢の一つです。

13.2.2　実装方法の改善

　実装レベルの観点で何よりも重要なのは、事業価値と実装方法の不一致がひどい箇所を見つけることです。たとえば、中核の業務領域の複雑な業務ロジックの実装に、トランザクションスクリプトやアクティブレコードを使っている場合です。中核の業務領域は、事業を存続し発展させるために変化し続けま

す。そして、この部分の設計がまずければ、変更はきわめてやっかいで危険になります。

13.2.3　同じ言葉を育てる

既存システムの設計改善を成功させる必要条件は、業務知識と適切なドメインモデルです。この本で何度も説明しているように、ドメイン駆動設計の根幹は、全員が使う同じ言葉です。業務知識を獲得し、役に立つ解決モデルを作るための根本となる手法が同じ言葉です。

業務知識を得る近道、つまりイベントストーミングを思い出してください。イベントストーミングを行って、業務エキスパートと協力して同じ言葉を育て、やっかいなコードベースの探検に取り組んでください。特に、ドキュメントのないコードベースを誰も理解できていない状況であれば、コードベースの探検が必要です。既存システムの業務機能に関わる人を集め、対象となっている業務領域を探検します。イベントストーミングは、失われた業務知識を取り戻す、すばらしい道具です。

業務知識とそのモデルが手に入ったら、対象となっている業務機能を実現するために、もっとも適切な業務ロジックの実装方法を選択してください。**第10章**で説明した経験則を使ってみましょう。その選択ができたら、改善の基本方針を選択します。つまり、システムの構成要素を段階的に置き換えていく方法（ストラングラー方式）にするか、既存の実装を段階的にリファクタリングしていくかの選択です。

13.2.3.1　ストラングラー方式

「ストラングラーフィグ」（絞め殺しの木）とは、**図13‐3**のような奇妙な生態を持つ熱帯植物です。ストラングラー（絞め殺す側）は、他の樹木（宿主）にくっついて育ちます。まずストラングラーの種が、宿主となる木の上部の枝にくっつきます。ストラングラーは下に向かって成長し、地面に届くとそこに根を下ろします。時間とともにストラングラーの葉が生い茂り、宿主となった木は光があたらなくなり、やがて枯れてしまいます。

図13-3　宿主の木を覆うストラングラーフィグ（絞め殺しの木）（出典：https://unsplash.com/photos/y_l5tep9wxI）

ストラングラー方式による移行は、この奇妙な木の生育のしかたを参考にしています。新たな要求を実装するための区切られた文脈をストラングラーとして追加し、既存システムの機能を少しずつそちらに移行します。既存システム側の区切られた文脈では、緊急度が高い修正を除き、機能の修正と拡張を停止します。最終的には既存システムのすべての機能を新規に追加した区切られた文脈に移行します。そして、現実のストラングラーと同じように、宿主となった既存システムのコードベースは死を迎えます。

ほとんどの場合、ストラングラー方式はファサード、つまり外部への公開インターフェースとして機能する薄い抽象レイヤーと組み合わせて使います。ファサードは外部からのリクエストを受け取って、そのリクエストを既存システムあるいは追加した区切られた文脈のどちらかに振り分けます。移行が終わったら、つまり既存システムの運用を停止したら、役目を終えたファサードを取り除きます（図13‐4）。

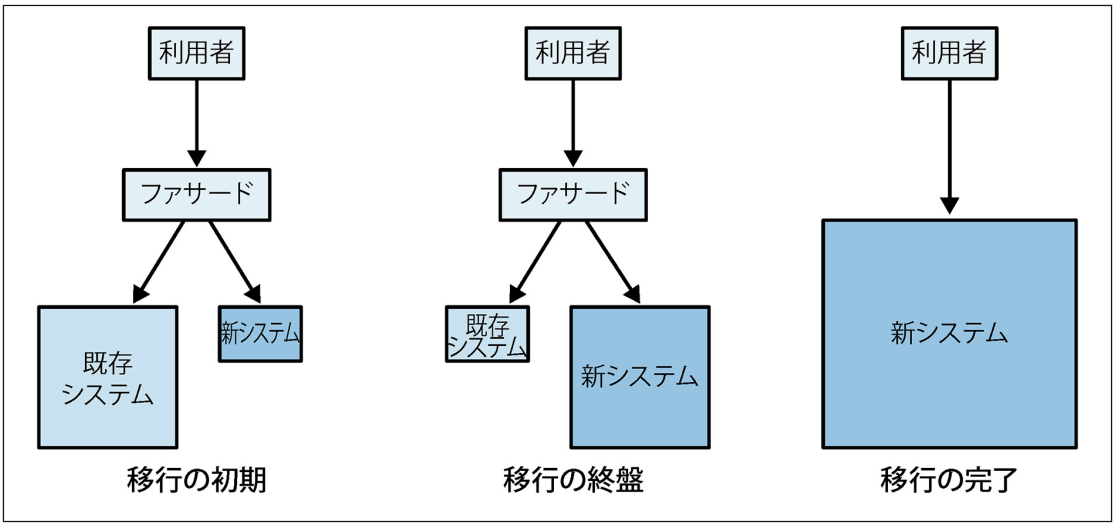

図13-4　移行段階に応じてファサードが外部からのリクエストを適切に振り分ける。移行が完了したら既存システムとファサードを取り除く

区切られた文脈は、本来は独立したサブシステムです。原則として、別の区切られた文脈とデータベースの共有はしません。しかしストラングラー方式で移行する場合は、この原則を緩めてもよいでしょう。新システムと既存システムとの連係を複雑にしないために、データベースを共有します。そうすることで、複数の区切られた文脈が同じデータを扱う分散トランザクションなどの複雑さを回避します（図13‐5）。

「一つの区切られた文脈に一つのデータベース」という原則をまげるのは、あくまでも一時的な対応です。既存システムは遅かれ早かれ消滅します。その結果、新システムの区切られた文脈が一つのデータベースを専有し、原則通りになります。

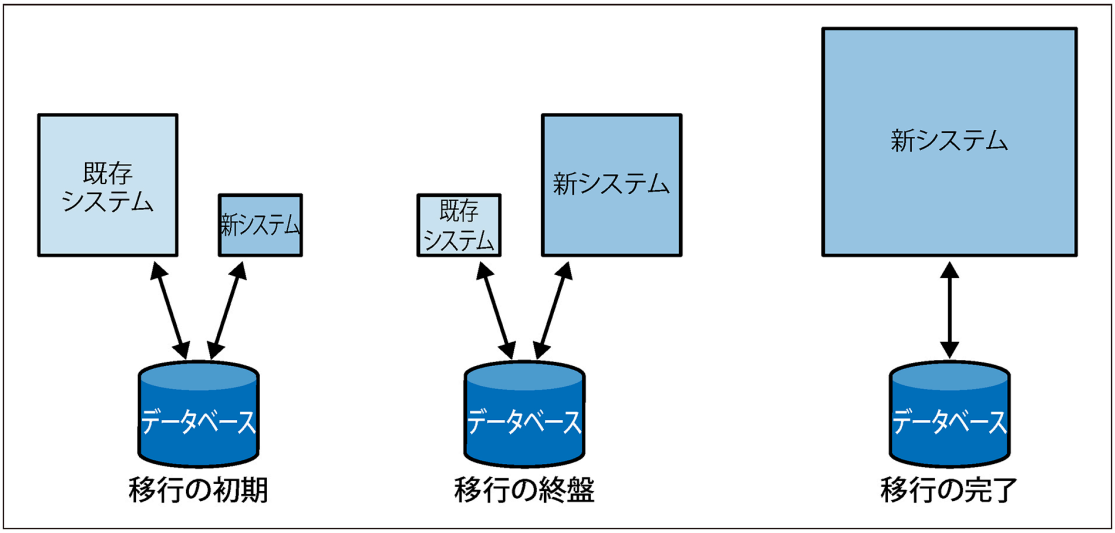

図13-5　既存システムと新システムで一時的にデータベースを共有する

移行方法として、ストラングラー方式とは別のやり方もあります。既存コードベースの設計改善、つまりリファクタリングによる移行です。

13.2.3.2　業務ロジックの実装方法のリファクタリング

第11章では、実装方法の変更についてさまざまな側面を検討しました。しかし既存のコードベースを対象に設計改善をする場合、二つの微妙な意味合いを意識する必要があります。

第一に、全面的な書き換えよりも、小さな変更を積み重ねるほうが安全だということです。つまり、トランザクションスクリプトやアクティブレコードを、いきなりイベント履歴式ドメインモデルに書き換えるのは無謀です。そうではなく、いったんは状態ありの集約を設計します。そして効果的な集約の境界を見つけることに時間を使います。関係する業務ロジックを確実に集約の境界の内側に配置します。いったん状態ありの集約に移行し、そこからイベント履歴式の集約へ移行するほうが、イベント履歴式の集約にまちがったトランザクション境界を持ち込む危険を冒すより、桁違いに安全です。

第二に、小さなステップを積み重ねるやり方であれば、ドメインモデルへの移行を一気に進める必要はありません。そのかわり、ドメインモデルを構成する部品を段階的に取り入れていけばよいのです。

値オブジェクトの候補を見つけるところから始めましょう。状態が変化しないオブジェクトを導入すると、プログラムの複雑さはかなり小さくなります。本格的なドメインモデルに移行しないとしても、十分な効果があります。

第11章で検討したように、アクティブレコードから集約への移行を一気に進める必要はありません。少しずつ段階的に進められます。まず、対象となる業務ロジックを集めます。そしてトランザクション処理の境界を詳しく調べます。結果的に整合すればよいデータに対して厳密な一貫性を保証する実装方法を選択していないでしょうか。既存のコードベースを分析する時に、技術的な視点ではなく、業務の視点で実装方法を選択することを忘れないでください。トランザクションの必要性を徹底的に分析することで、初めて集約の境界を設計できます。

最後に、既存システムをリファクタリングする場合、必要であれば、モデル変換装置を使って新しいコードベースを古いモデルから保護し、共用サービス

を作って公開された言葉を提供することで、上流のコンポーネントの変更が下流のコンポーネントに影響しないようにしてください。

13.3　実践的なドメイン駆動設計

この章の最初で検討したように、ドメイン駆動設計の適用とは「すべてをやる」か「まったくやらない」かの二者択一ではありません。ドメイン駆動設計が提供するすべての道具を使う必要はありません。たとえば、何らかの理由で実装レベルの設計のやり方は、個別の状況には役に立たないかもしれません。今取り組んでいる領域では、もっと別の設計方法が好ましいかもしれません。ドメイン駆動設計とは異なる別の設計方法が効果的だと気づくこともあるでしょう。それでもよいのです。

対象としている業務領域と事業方針を詳しく調べ、課題の解決に役立つモデルを探してソフトウェアを設計してください。もっともたいせつなことは、事業活動の要求にもとづいた設計判断です。それがドメイン駆動設計です。

繰り返しますが、集約や値オブジェクトの実装がドメイン駆動設計ではありません。ソフトウェアの「設計」判断を「事業活動（ドメイン）」で駆動するのがドメイン駆動設計です。

13.4　ドメイン駆動設計を売り込む

技術カンファレンスでドメイン駆動設計について発表すると、ほぼ毎回される質問があります。「すばらしい内容でした。しかし、チームや経営層にどうやったらドメイン駆動設計を売り込めるでしょう？」これはきわめて重要な質問です。

ドメイン駆動設計を売り込むことは難しいでしょう。個人的には、「売り込む」という考え方は嫌いです。どういうことかというと、そういう考え方をすると、設計活動ではなく、設計方法の営業活動になってしまうからです。ソフトウェア技術者は、設計した結果を、チーム、経営陣、そして顧客に提供します。しかし、設計の幅広い側面をカバーし、技術領域の外側にいる利害関係者を巻き込むような設計方法論を、何か価値のある商品として誰かに説明して売り込むことは、きわめて困難でしょう。

組織に大きな変革を持ち込むには、経営陣のサポートが不可欠です。しかし、多くの経営陣はドメイン駆動設計を知らないし、ドメイン駆動設計の事業

的な価値を学ぼうとしません。ドメイン駆動設計は、経営陣にとって、最優先の課題ではありません。ましてや、ドメイン駆動設計のように組織の開発のあり方を大きく変える取り組みは、経営者にとって魅力的な選択肢ではありません。しかし、幸いなことに、そういう状況でもドメイン駆動設計に取り組むことは可能です。

13.4.1　ドメイン駆動設計を非公式に取り入れる

ドメイン駆動設計を技術者が使う道具の一つと位置づけましょう。組織戦略として取り入れる必要はありません。ドメイン駆動設計の考え方とやり方は技術手法です。ソフトウェア設計は技術者の仕事です。ですから、自分たちの仕事に使う便利な道具として、ドメイン駆動設計を取り入れてください。

周囲に売り込む方法ではなく、ドメイン駆動設計を日々の仕事にどのように活用すればよいかを見てみましょう。

13.4.1.1　同じ言葉

同じ言葉を使うことが、ドメイン駆動設計に取り組む土台です。同じ言葉は、業務知識を発見し、お互いの意図を伝え、役に立つモデルを作るために絶対に必要です。

幸いなことに、同じ言葉はしごくあたりまえのやり方です。事業活動について関係者が口にする言葉をしっかりと聞き取ってください。技術的な用語を使う場合、技術者だけに通じる特殊な言い回しを避け、業務的に意味が通じる言葉の使い方をしてください。

使い方が一貫しない用語を見つけ、質問をしてみましょう。たとえば、一つの対象に異なる表現が使われていたら、その理由を見つけてください。一つのソフトウェアの中で異なるモデルが混在しているなら、言葉の意味を決める文脈を特定してください。複数の言葉が同じ意味で使われていたら、その共通の意味を表現する一つの用語を使うように提案しましょう。

また、業務エキスパートとできるだけたくさんの会話をしてください。正式な会議や打ち合わせだけが会話の機会ではありません。ウォーターサーバーの前やコーヒーブレークの時間は、ちょっとした会話のよいきっかけとなるでしょう。業務エキスパートと事業活動について会話してみましょう。業務の言葉を使ってみましょう。自分たちの理解があやふやなところを見つけ、その点に

ついて説明をお願いしてみましょう。心配いりません。開発対象の業務を本気で学ぼうとする技術者と一緒に仕事をすることは、たいていの業務エキスパートにとって、うれしいことです。

もっともたいせつなことは、同じ言葉を使ってソースコードを記述し、ソフトウェア開発のあらゆる意図の伝達に同じ言葉を使うことです。忍耐強く続けてください。組織の中で長く使われてきた用語を変えることは時間はかかりますが、いずれは浸透するでしょう。

13.4.1.2　区切られた文脈

全体をどう分割するかを検討する時は、区切られた文脈の背景にある原則にこだわりましょう。つまり、以下の内容です。

- あらゆるユースケースを一つのモデルで表現するよりも、課題ごとに複数のモデルを作るとよいのはなぜでしょうか？その答えは、「何でもできる」は実際には何の役にも立たないからです。
- 一つの区切られた文脈の中に複数のモデルを作らないのはなぜでしょうか？その答えは、異なるモデルが混在すると、認知の負荷が大きくなり、ソフトウェアが不必要に複雑になるからです。
- 同じコードベースを複数のチームで開発するのがよくないのはなぜでしょうか？その答えは、チーム間の関係がぎくしゃくし、仕事がうまく進められないからです。

区切られた文脈どうしの連係方法を検討する時も、その背景にある原則にこだわってください。つまり、それぞれの連係方法がどういう課題を解決しようとしているかをきちんと理解して、適切な方法を選択してください。

13.4.1.3　業務ロジックの実装方法

実装方法を検討する時に、権威に訴えてはいけません。つまり「ドメイン駆動設計の本にそう書いてあるから集約を使おう」という説得のしかたではいけません。そうではなくて、論理で説得しましょう。たとえば、次のような説明です。

- トランザクション境界を明確にすることが重要なのはなぜか？データの一貫性を維持するため。

- データベースのトランザクション処理で、一つ以上の集約インスタンスを同時に変更してはいけないのはなぜか？データの一貫性を維持する境界を、正しく定義することを保証するため。
- 集約内部の状態を、集約の外部から直接変更してはいけないのはなぜか？関連する業務ロジックを1箇所に集め、重複させないため。
- 集約の機能の一部をストアードプロシージャに移動してはいけないのはなぜか？ロジックの記述を重複させないため。重複したロジック、特に物理的にも論理的にも離れたコンポーネントに重複したロジックは、実装に差異が生まれやすく、データ不整合の原因になりがち。
- 集約をできるだけ小さく保つべき理由は何か？トランザクション単位を大きくすると、集約が不必要に複雑になるから。処理性能も悪化する。
- イベントソーシングのかわりにログファイルを利用してはいけないのはなぜか？長期的に見た時に、ログファイルではデータの一貫性を保証できないから。

イベントソーシングについて言えば、イベント履歴式ドメインモデルが必要な場合、この実装方法を関係者に説明し理解してもらうことは難しいでしょう。こういう場合に役に立つのがジェダイのマインドトリックです。それを見てみましょう。

13.4.1.4　イベント履歴式ドメインモデル

イベントソーシングは多くの利点があるにもかかわらず、たいていの人は極端すぎると感じるようです。この本で一貫して検討してきたように、この解決方法についても、事業の観点で判断してください。

業務エキスパートと会話をします。状態更新式とイベント履歴式の2種類のモデルがあることを伝えます。二つのやり方の違いと、イベントソーシングの利点を説明します。特に、時間軸にそったモデルの利点を説明します。たいていの場合、イベントソーシングから得られる洞察の深さをとても気に入り、イベントソーシングを同僚に積極的に売り込んでくれるでしょう。

業務エキスパートと話す時は、同じ言葉を使うことを絶対に忘れないでください。

13.5　まとめ

この章ではドメイン駆動設計の考え方とやり方を現実のソフトウェア開発にどう取り込んでいくかを学びました。つまり、変更がやっかいで危険な既存システムのコードベースを対象に、開発チームが必ずしもドメイン駆動設計に習熟していない状況での取り組み方です。

新規の開発プロジェクトと同じように、既存システムに取り組むプロジェクトでも、事業活動を詳しく調べるところから始めてください。自社の事業目標は何でしょうか。目標を達成するための事業方針はどうなっているでしょうか。事業活動がどのような業務領域に分かれ、それぞれの業務領域のカテゴリーが何であるかを特定するために、組織図と既存システムの設計内容を参考にしてください。そこから明らかになったことを元にして、システム全体の改善方針を検討します。重点的に解決すべき課題を特定します。事業価値を生み出す源泉はどこでしょうか。既存コードのリファクタリング、あるいは、システムの構成要素の置き換えによってシステムを改善してください。どちらの方法であっても、少しずつ段階的に進めましょう。全面的な書き換えは、期待できる事業価値よりも失敗のリスクが上回ります。

最後に。ドメイン駆動設計が組織に受け入れられていない状況でも、ドメイン駆動設計に取り組むことは可能です。適切な道具を選んで使ってください。そしてドメイン駆動設計のやり方をチームのメンバーと議論をする時は、それぞれのやり方の背景にある考え方と原則を意識して使ってください。

ドメイン駆動設計そのものの検討はこの章でいったん終わります。次の**第IV部**では、ドメイン駆動設計とその他の方法論や設計技法との関係を学びます。

13.6　演習問題

1. ドメイン駆動設計の考え方とやり方を既存システムのプロジェクトに取り入れる時、最初にやるべきことは何ですか？
   A. すべての業務ロジックをイベント履歴式ドメインモデルに書き換える
   B. 自社の事業活動と事業方針を詳しく調べる
   C. 既存システムの構造を区切られた文脈の考え方で改善する
   D. 既存システムにドメイン駆動設計を取り入れることは不可能
2. 既存システムを適切な構造に改善するためにストラングラー方式で取り組む場合、ドメイン駆動設計の設計原則と矛盾するのはどういう場合ですか？
   A. 複数の区切られた文脈が一つのデータベースを共有する

B. 中核の業務領域で、移行用に追加した区切られた文脈の機能と既存システムの機能で業務ロジックが重複する

C. 複数のチームで一つの区切られた文脈を開発する

D. AとB

3. たいていの場合、アクティブレコードで記述している業務ロジックをイベント履歴式ドメインモデルに直接移行することは、よいアイデアではありません。その理由は何ですか？

A. いったん状態変更ありのモデルに移行することで、集約の境界のリファクタリングを学習しながら進められる

B. 段階的に移行していくほうが安全だから

C. AとB

D. どれもあてはまらない。アクティブレコードからはもちろん、トランザクションスクリプトからでも、イベント履歴式ドメインモデルに一気に書き換えるのが適切なやり方となる

4. 集約を取り入れる時、チームのメンバーから次の質問をされました。「できるだけ多くのエンティティを一つの集約にまとめれば、業務ロジックが1箇所に集まり、処理が追いかけやすい。そうしない理由は？」 あなたなら、どう説明しますか？